

Probability & Statistics (MT-2005)

Descriptive Statistics

M Ali, S Aziz, S Hassan

Tentative Marks Distribution & Text Books

Assessment Type	Weightages
Quiz	10%
Homeworks	15%
Sessional I	15%
Sessional II	15%
Final Exam	45%

* Absolute grading policy would be followed

Text Books

- Elementary Statistics, A step by Step Approach, Allan G. Bluman
- Probability and Statistics for Engineers and Scientists by Walpole, Myers, and Myers.

Reference Books

- Introduction to Probability by Dimitri P. Bertsekas and John N. Tsitsiklis.
- Think Stats: Exploratory Data Analysis in Python by Allen B. Downey.
- The Elements of Statistical Learning by Hastie, Tibshirani, and Friedman

What is Probability and Statistics

Probability

Probability is a measure of the likelihood that an event will occur, expressed as a value between 0 and 1, where:

- 0 means the event is impossible.
- 1 means the event is certain.

It is mathematically given by:

$$P = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

Statistics

Statistics is the branch of mathematics that involves collecting, organizing, analyzing, interpreting, and presenting data to draw meaningful conclusions and make informed decisions.

What is Probability and Statistics cont'd...

Difference Between Probability and Statistics

- **Probability:** Starts with a model to predict the likelihood of future outcomes (predictive, theory-to-data).
- **Statistics:** Starts with data to infer parameters or understand the underlying model (inferential, data-to-theory).

Similarities Between Probability and Statistics

Both deal with uncertainty and use mathematical frameworks to analyze and make decisions about random phenomena. They rely on concepts like events, distributions, and outcomes.

Branches of Statistics

The study of statistics has two major branches depending on how data are used: **descriptive statistics** and **inferential statistics**.

- ① **Descriptive Statistics:** Focuses on summarizing and describing the main features of a data set. Methods include:
 - **Data visualization:** Tables, Graphs, charts.
 - **Numerical descriptive measures:** Measures of central tendency, Measures of variability, Measures of position.
- ② **Inferential Statistics:** Concerned with making predictions or inferences about a population based on a sample. It involves techniques such as:
 - **Hypothesis testing.**
 - **Confidence intervals.**
 - **Regression analysis.**

Inferential statistics uses probability theory to estimate population parameters.

Essential Statistical Terminologies

- **Population:** The entire group of individuals, items, or events that you want to study or make inferences about.
- **Sample:** A subset of the population selected for analysis, intended to represent the population.
- **Parameter:** A numerical value that describes a characteristic of a population.
- **Statistic:** A numerical value that describes a characteristic of a sample.
- **Variable:** A symbol or placeholder that can take any value from a set of possible values.
- **Random Variable:** Variables whose values are determined by chance.
- **Data:** Values (measurements or observations) that the variables can assume.

Essential Statistical Terminologies cont'd...

- **Deterministic Model:** Always produces the same outcome for a given input. No randomness or uncertainty involved.
 - Example: $s = ut + \frac{1}{2}at^2$ (predicts exact position of an object under uniform acceleration).
- **Probabilistic Model:** Involves uncertainty and describes outcomes using probabilities.
 - Example: Predicting the weather, with $P(\text{Rain}) = 0.7$ and $P(\text{No Rain}) = 0.3$.
- **Accuracy:** Accuracy refers to how close a measured value is to the true value. Accuracy is concerned with the systematic errors or biases in measurements or predictions.
- **Precision:** Precision, on the other hand, refers to the consistency or repeatability of the measurements, or how close the measured values are to each other. Precision focuses on the random errors and how spread out the measurements are.

- **Outlier:** A data point or observation that differs significantly from other observations in a dataset. It is an unusually large or small value that does not fit the general pattern or trend of the data.

Data and Types of Data

A collection of facts, numbers, or observations analyzed to extract useful information.

- **By Nature:**

- **Qualitative (Categorical):**

- **Nominal:** Categories without order (e.g., Gender: Male, Female).
 - **Ordinal:** Ordered categories without equal intervals (e.g., Education Level: High School < Bachelor < Master).

- **Quantitative (Numerical):**

- **Discrete:** Countable values (e.g., Number of students).
 - **Continuous:** Measurable values (e.g., Height, Temperature).

Levels of Measurement for Numerical Data

- **Interval:**

- Ordered, equal intervals, no true zero.
- Examples: Temperature, IQ scores.

- **Ratio:**

- Ordered, equal intervals, true zero exists.
- Examples: Height, Time.

- **Univariate Data:**

- Single variable focus (e.g., Heights of students).
- Analysis: Mean, Median, SD.

- **Bivariate Data:**

- Two variables, relationship analysis (e.g., Height vs Weight).
- Techniques: Correlation, Regression.

- **Multivariate Data:**

- More than two variables (e.g., Age, Height, Income).
- Techniques: PCA, MANOVA.

- **Rectangular Data:**

- Organized in tables (e.g., Students' grades).

- **Non-Rectangular Data:**

- Unstructured/Semi-structured (e.g., Emails, Hierarchical data).

- **Cross-Sectional Data:**

- Differences across entities at a single time point. (e.g., Income of people in 2024).

- **Time-Series Data:**

- Trends and patterns over time for one entity. (e.g., Apple stock price (2000-2024)).

- **Panel Data:**

- Combines cross-sectional and time-series data.
- Differences across entities and trends over time. (e.g., Income of people tracked from 2000 to 2024).

Collection of Data & Sampling Techniques

Collection of Data & Sampling Techniques

Data Collection Methods

- **Primary:** Collected directly from experiments or surveys/interviews.
- **Secondary:** Extracted from existing sources (e.g., government databases).

Sampling Techniques

Statisticians use four basic methods of sampling:

1. **Random Sampling:** Each individual in the population has an equal chance of being selected.
 - **Method:** Random selection is typically achieved using a random number generator or drawing lots.
 - **Example:** Selecting 10 students randomly from a list of 100 students.
2. **Systematic Sampling:** Selects individuals at regular intervals from a sorted list.
 - **Method:** Choose a starting point randomly, then pick every k -th individual (e.g., every 5th person).
 - **Example:** In a group of 1,000, choosing every 10th person after randomly selecting a starting point.

3. Stratified Sampling: The population is divided into subgroups (strata) based on a characteristic, and random samples are drawn from each stratum.
 - **Method:** Proportional or equal representation is ensured from each subgroup.
 - **Example:** Surveying students by dividing them into strata based on grades.
4. Cluster Sampling: The population is divided into clusters, and a random selection of clusters is made. All individuals in the selected clusters are included in the sample.
 - **Method:** Often used when the population is geographically dispersed.
 - **Example:** Selecting schools randomly in a district and surveying all students in the chosen schools.

Representation of Data

Data representation is crucial for understanding sample data. Two key components of data representation:

- ① **Data Organization:** Organizing data helps to visualize patterns and analyze relationships.
 - The Categorical Frequency Distribution, The Contingency Table. (For Qualitative Data)
 - The Frequency Distribution (Grouped, Ungrouped). (For Quantitative Data)
- ② **Data Visualization:** Visualizing data reveals patterns and relationships.
 - Bar Chart, Multiple Bar Chart, Component Bar Chart, Pie Chart, etc. (For Qualitative Data)
 - Histogram, Frequency Polygon, Ogive, etc. (For Quantitative Data)

Qualitative Data: Organization & Visualization

Organization: Categorical Frequency Distribution

A **categorical frequency distribution** is a method of organizing data into categories or groups and counting how many observations fall into each category. This type of distribution is particularly useful for summarizing data measured on a nominal or ordinal scale, such as gender, color, brand preference, or other qualitative attributes.

Components of a Categorical Frequency Distribution

- **Category (Class):** These are the distinct groups or categories of the data.
- **Tally:** A quick and visual method of counting data, where each observation is represented by a mark (e.g., ||||).
- **Frequency:** The count of how many observations fall into each category.
- **Relative Frequency** (optional): The proportion or percentage of the total that each category represents.
- **Cumulative Frequency** (optional): The running total of frequencies up to a certain category.

Example # 1: The Categorical Frequency Distribution

The following list represents the recorded software errors observed during testing:

- **S:** Syntax Errors
- **R:** Runtime Errors
- **L:** Logical Errors

{S, R, S, L, R, S, S, L, R, S, L, R, S, S, S, R, L, S, S, R, R, S, L,
S, R, L, R, S, R, S, L, L, S, S, R, S, R, L, R, S, R, L, S, L, S,
R, L, S, L, S, R, L, R, R, L, S, L, S, S}

Make the table that summarizes the frequency distribution of three types of software errors along with their tally, relative frequencies percentages, where

$$\text{Relative Frequency Percentages} = \frac{\text{Frequency of a Category}}{\text{Total Frequency}} \times 100$$

Organization: Categorical Frequency Distribution Cont'd...

Error Type	Tally	Frequency	Cumulative Frequency	Percentage
Syntax Errors	, , , ,	25	25	41.67%
Runtime Errors	, , ,	20	45	33.33%
Logical Errors	, , ,	15	60	25.00%
Total	-	60		100.00%

Table: Frequency Distribution of Software Errors

Visualization: Simple Bar Chart

A **simple bar graph** (also called a bar chart) is a graphical representation of **categorical data** where each category is represented by a rectangular bar. The height or length of each bar is proportional to the value it represents. The data in **Example # 1** can be represented by using simple bar chart as:

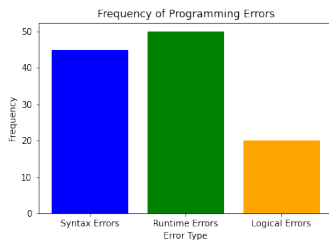


Figure: Errors Across Operating Systems

Visualization: Python Code for Simple Bar Chart

Here is the Python code for creating a bar chart of **Example # 1**:

```
import matplotlib.pyplot as plt

# Data for bar chart
error_types = ['Syntax Errors', 'Runtime Errors', 'Logical Errors']
frequencies = [45, 50, 20]

# Create bar chart
plt.bar(error_types, frequencies, color=['blue', 'green', 'orange'])
plt.title('Frequency of Programming Errors')
plt.xlabel('Error Type')
plt.ylabel('Frequency')

# Save the plot
plt.savefig('bar_chart.png')
plt.close()
```


Organization: Contingency Table

Contingency table is a type of data table that summarizes the relationship between two categorical variables. Each cell in the table represents the frequency (or count) of observations for a specific combination of the variables.

Example # 2: Errors by Operating System

The table below shows the distribution of different types of programming errors across various operating systems during testing:

Error Type / OS	Windows	Linux	MacOS	Total
Syntax Errors	20	15	10	45
Runtime Errors	10	25	15	50
Logical Errors	5	10	5	20
Total	35	50	30	115

Table: Errors by Operating System

Visualization: Multiple Bar Chart

A **multiple bar graph** (also called a grouped bar chart) is a graphical representation of categorical data where each category is represented by a group of bars. Each bar in the group corresponds to a subcategory, and the height or length of the bars is proportional to the values they represent. This type of chart is useful for comparing multiple datasets across the same categories.

The multiple bar chart of **Example # 2** is

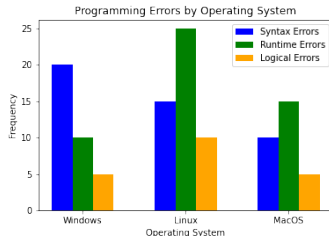


Figure: Errors Across Operating Systems

Visualization: Python Code for Multiple Bar Chart

Here is the Python code for creating a multiple bar chart of Example # 2:

```
import matplotlib.pyplot as plt
import numpy as np

# Data for multiple bar chart
os_types = ['Windows', 'Linux', 'MacOS']
syntax_errors = [20, 15, 10]
runtime_errors = [10, 25, 15]
logical_errors = [5, 10, 5]

# Bar width and positions
bar_width = 0.2
x = np.arange(len(os_types))
```

Visualization: Python Code for Multiple Bar Chart cont'd...

```
# Plot multiple bars
plt.bar(x - bar_width, syntax_errors, width=bar_width, label=
        'Syntax Errors', color='blue')
plt.bar(x, runtime_errors, width=bar_width, label='Runtime
        Errors', color='green')
plt.bar(x + bar_width, logical_errors, width=bar_width,
        label='Logical Errors', color='orange')

# Customize chart
plt.title('Programming Errors by Operating System')
plt.xlabel('Operating System')
plt.ylabel('Frequency')
plt.xticks(x, os_types)
plt.legend()

# Save the plot
plt.savefig('multiple_bar_chart.png')
plt.close()
```

Visualization: Component/Stacked Bar Chart

A **component bar graph** (also called a stacked bar chart) is a graphical representation of categorical data, where each category is represented by a single bar divided into segments. Each segment represents a subcategory, and the height (or length) of the bar is proportional to the total value of all segments combined. This type of chart is useful for visualizing the composition of categories and comparing totals.

The following Component/Stacked Bar Chart of **Example # 2**:

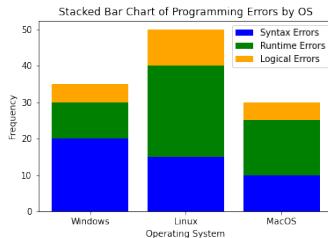


Figure: Stacked Bar Chart: Errors Across Operating Systems

Visualization: Python Code for Component/Stacked Bar Chart

The python code for creating Component/Stacked Bar Chart of **Example # 2** is:

```
import matplotlib.pyplot as plt
import numpy as np

# Data for stacked bar chart
os_types = ['Windows', 'Linux', 'MacOS']
syntax_errors = [20, 15, 10]
runtime_errors = [10, 25, 15]
logical_errors = [5, 10, 5]

# Plot stacked bars
plt.bar(os_types, syntax_errors, label='Syntax Errors',
        color='blue')
```

Visualization: Python Code for Component/Stacked Bar Chart cont'd...

```
plt.bar(os_types, runtime_errors, bottom=syntax_errors,
        label='Runtime Errors', color='green')
plt.bar(os_types, logical_errors, bottom=np.add(
        syntax_errors, runtime_errors), label='Logical Errors',
        color='orange')
# Customize chart
plt.title('Stacked Bar Chart of Programming Errors by OS')
plt.xlabel('Operating System')
plt.ylabel('Frequency')
plt.legend()

# Save the plot
plt.savefig('stacked_bar_chart.png')
plt.close()
```

Visualization: Pie Chart

A **pie chart** is a circular graph divided into slices to illustrate numerical proportions. Each slice represents a category or part of a whole, with the size of each slice corresponding to the percentage or proportion of that category relative to the entire dataset.

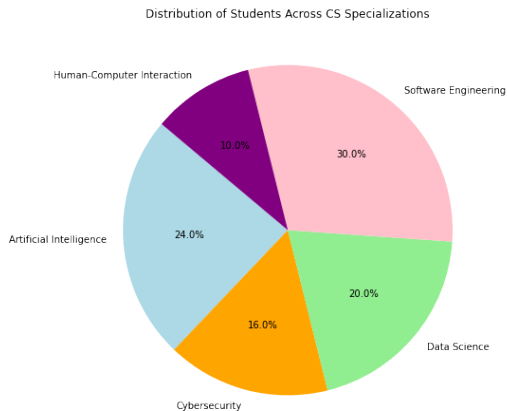
Example # 3: The table below shows the distribution of students across various Computer Science specializations:

Specialization	Number of Students
Artificial Intelligence	120
Cybersecurity	80
Data Science	100
Software Engineering	150
Human-Computer Interaction	50
Total	400

Table: Distribution of Students Across CS Specializations

Visualization: Pie Chart cont'd...

The following is Pie Chart for **Example # 3**:



Visualization: Python Code for Pie Chart

The python code for creating Pie Chart of **Example # 3** is:

```
import matplotlib.pyplot as plt

# Data
specializations = ['Artificial Intelligence', 'Cybersecurity',
                   'Data Science',
                   'Software Engineering', 'Human-Computer Interaction']
students = [120, 80, 100, 150, 50]
colors = ['lightblue', 'orange', 'lightgreen', 'pink', 'purple'] # Custom colors

# Create the pie chart
plt.figure(figsize=(8, 8))
plt.pie(students, labels=specializations, autopct='%1.1f%%',
        startangle=140, colors=colors)
```

Visualization: Python Code for Pie Chart cont'd...

```
# Add title
plt.title('Distribution of Students Across CS
          Specializations')

# Save the plot
plt.savefig('pie.png')
plt.close()
```

Visualization: Pareto Chart

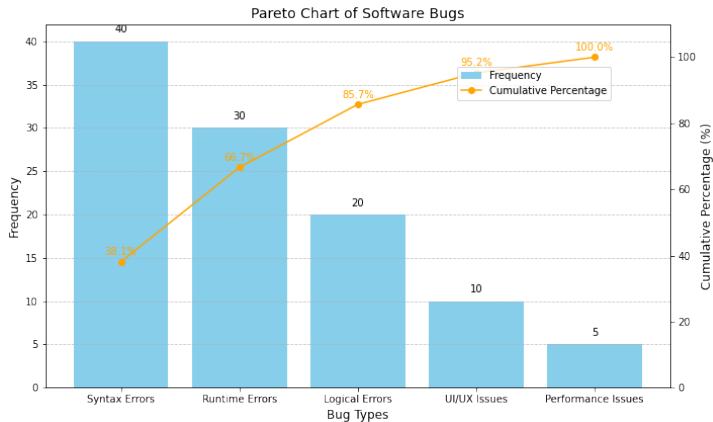
A Pareto chart is a combination of a bar chart and a line graph, where the bars represent frequencies (or counts), and the line represents the cumulative percentage. It's commonly used to identify the most significant factors in a dataset.

Example # 4: The following data represents the frequency of various types of software bugs during a project:

Bug Type	Number of Bugs
Syntax Errors	40
Runtime Errors	30
Logical Errors	20
UI/UX Issues	10
Performance Issues	5
Total	400

Visualization: Pareto Chart cont'd...

The following is Pareto Chart for Example # 4:



Visualization: Python code for Pareto Chart

The python code for creating Pareto Chart of **Example # 4** is:

```
import matplotlib.pyplot as plt
import numpy as np

# Data
bug_types = ['Syntax Errors', 'Runtime Errors', 'Logical
            Errors', 'UI/UX Issues', 'Performance Issues']
frequencies = [40, 30, 20, 10, 5]

# Sort data in descending order (if not already sorted)
sorted_indices = np.argsort(frequencies)[::-1]
bug_types = [bug_types[i] for i in sorted_indices]
frequencies = [frequencies[i] for i in sorted_indices]

# Calculate cumulative percentages
cumulative_percentages = np.cumsum(frequencies) / sum(
    frequencies) * 100
```

Visualization: Python code for Pareto Chart cont'd...

```
# Create the bar chart
fig, ax1 = plt.subplots(figsize=(10, 6))
bars = ax1.bar(bug_types, frequencies, color='skyblue',
               label='Frequency')
ax1.set_ylabel('Frequency', fontsize=12)
ax1.set_xlabel('Bug Types', fontsize=12)
ax1.set_title('Pareto Chart of Software Bugs', fontsize=14)

# Add cumulative percentage line
ax2 = ax1.twinx()
ax2.plot(bug_types, cumulative_percentages, color='orange',
         marker='o', label='Cumulative Percentage')
ax2.set_ylabel('Cumulative Percentage (%)', fontsize=12)
ax2.set_ylim(0, 110)
```

Visualization: Python code for Pareto Chart cont'd...

```
# Add gridlines and annotations
ax1.grid(axis='y', linestyle='--', alpha=0.7)
for i, bar in enumerate(bars):
    ax1.text(bar.get_x() + bar.get_width() / 2, bar.
        get_height() + 1, f'{frequencies[i]}', ha='center',
        fontsize=10)
for i, pct in enumerate(cumulative_percentages):
    ax2.text(i, pct + 2, f'{pct:.1f}%', ha='center',
        fontsize=10, color='orange')

# Show legend
fig.legend(loc='upper right', bbox_to_anchor=(0.85, 0.85),
    fontsize=10)
plt.tight_layout()

# Save the plot
plt.savefig('pareto.png')
plt.close()
```


Quantitative Data: Organization & Visualization

Organization: Frequency Distribution for Ungrouped Data

An **ungrouped frequency distribution** organizes data into individual values (rather than grouped intervals) and shows the number of times each value occurs. This method is commonly used when the dataset contains a relatively small range of distinct values, making it feasible to list all individual values.

Example # 5: Consider the marks obtained by 15 students in a test:

5, 7, 8, 5, 6, 8, 7, 9, 6, 7, 5, 8, 6, 7, 9

The ungrouped frequency distribution table is as follows:

Marks (Value)	Tally	Frequency	Relative Frequency (%)	Cumulative Frequency
5		3	$\frac{3}{15} \times 100 = 20$	3
6		3	$\frac{3}{15} \times 100 = 20$	6
7		4	$\frac{4}{15} \times 100 \approx 26.67$	10
8		3	$\frac{3}{15} \times 100 = 20$	13
9		2	$\frac{2}{15} \times 100 \approx 13.33$	15

Table: Frequency Distribution of Marks

Organization: Frequency Distribution for Grouped Data

A **grouped frequency distribution** organizes data into intervals (called *classes*) instead of listing individual values. This approach is particularly useful for large datasets with a wide range of values, as it provides a concise summary of the data.

Key Concepts in Grouped Frequency Distribution

- **Range:** The difference between the largest and smallest values in the dataset:

$$\text{Range} = \text{Maximum Value} - \text{Minimum Value}$$

- **Number of Classes:** Sturges' rule gives the number of classes (k) for grouped data:

$$k = 1 + 3.322 \log_{10} n$$

where n is the number of data points.

Organization: Frequency Distribution for Grouped Data

Cont'd...

- **Class Intervals:** Non-overlapping intervals into which data is grouped.
- **Class Limits:** The smallest and largest values within each class interval.
 - **Lower Class Limit:** The smallest value in a class.
 - **Upper Class Limit:** The largest value in a class.
- **Class Boundaries:** The true boundaries between classes, calculated by averaging the upper limit of one class and the lower limit of the next class. Class boundaries prevent gaps between classes.

Organization: Frequency Distribution for Grouped Data

Cont'd...

- **Class Width:** The difference between the lower limits of two consecutive classes or the difference between the upper and lower boundaries of a class. Class width can be calculated as:

$$\text{Class Width} = \frac{\text{Range}}{\text{number of classes}}$$

- **Midpoint:** The central value of a class interval, calculated as:

$$\text{Midpoint} = \frac{\text{Lower Class Limit} + \text{Upper Class Limit}}{2}$$

Organization: Frequency Distribution for Grouped Data

Cont'd...

Example # 6: The scores of 30 students in a mathematics exam are given below. Represent the data distribution using a histogram.

Scores of Students
75, 88, 92, 70, 81, 79, 85, 73, 95, 68, 82, 78, 80, 72, 77, 84, 69, 74, 83, 71, 90, 87, 76, 89, 93, 86, 91, 94, 67, 66

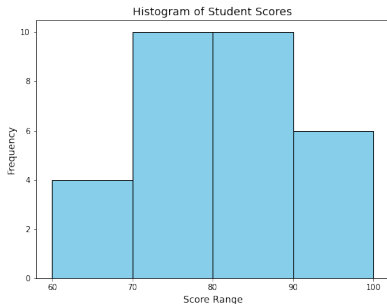
The scores of 30 students in a mathematics exam are grouped into score ranges. Below is the frequency table showing the distribution of scores:

Score Range	Frequency	Midpoint	Cumulative Frequency
60–70	6	65	6
70–80	10	75	16
80–90	8	85	24
90–100	6	95	30

Visualization: Histogram

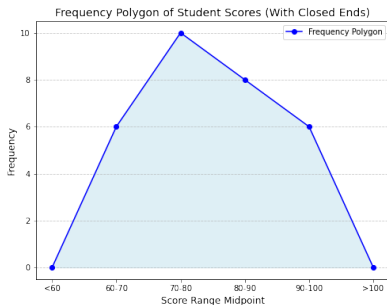
The **histogram** is a graph that displays the data by using contiguous vertical bars (unless the frequency of a class is 0) of various heights to represent the frequencies of the classes.

Below is the Histogram for **Example # 6**:



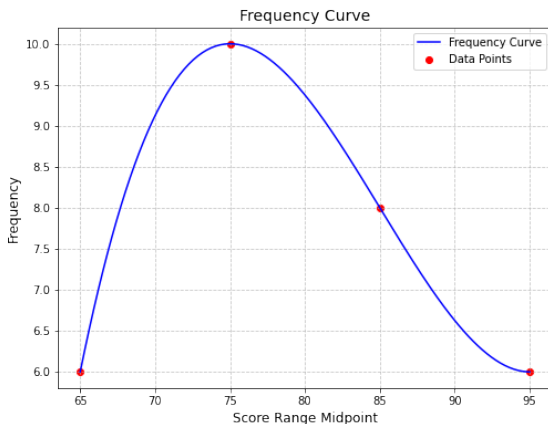
Visualization: Frequency Polygon

The **frequency polygon** is a graph that displays the data by using lines that connect points plotted for the frequencies at the midpoints of the classes. The frequencies are represented by the heights of the points. Below is the Frequency Polygon for **Example # 6**:



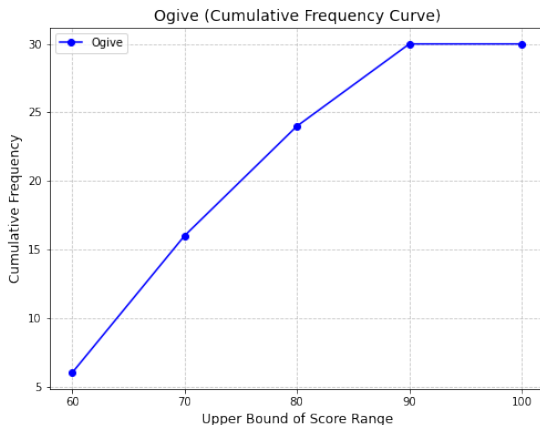
Visualization: Frequency Curve

A **frequency curve** is a smooth curve that represents the distribution of data by connecting the tops of histogram bars or frequency polygon points. Below is the Frequency Curve for **Example # 6**:



Visualization: Ogive

The **ogive** is a graph that represents the cumulative frequencies for the classes in a frequency distribution. Below is the Ogive for **Example # 6**:



Visualization: Time Series Graph

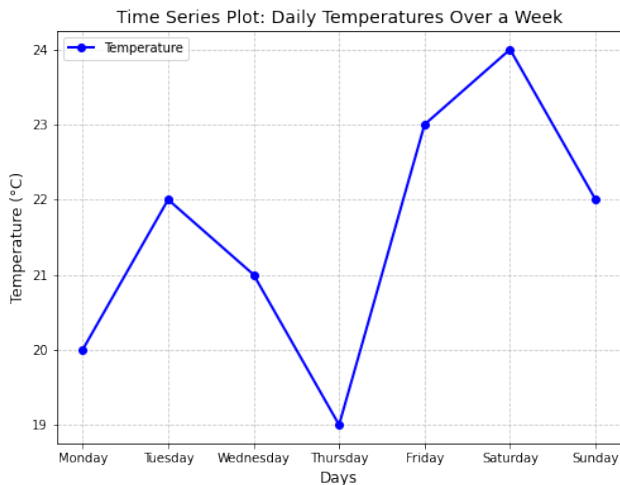
A **time series graph** represents data that occur over a specific period of time.

Example # 9: The following table shows the daily temperatures (in $^{\circ}\text{C}$) recorded over a week:

Day	Temperature ($^{\circ}\text{C}$)
Monday	20
Tuesday	22
Wednesday	21
Thursday	19
Friday	23
Saturday	24
Sunday	22

Visualization: Time Series Graph Cont'd...

Below is the Time Series graph for **Example # 9**:



Visualization: Scatter Plots

A **scatter plot** is used to visualize the relationship between two numerical variables. Each point represents an observation, with the x-axis and y-axis corresponding to the two variables.

Example # 10: The following table shows hours studied vs. exam scores:

Hours Studied	Exam Scores
4.2	65
4.8	70
5.5	75
6.1	78
6.7	82
7.3	85
7.9	88
8.0	90

Visualization: Scatter Plots Cont'd...

Below is the Scatter plot for **Example # 10**:

